Batch 20 — Supplier + Storekeeping Deepening Pack

Daniel Macharia <danielmacharia43@gmail.com>
To: <daniel@ics.ac.ke>

Tue, 16 Jun at 08:22

Batch 20 — Supplier + Storekeeping Deepening Pack

Meat Lovers CIMS powered by YohPal

20A.

database/migrations/033_supplier_invoices.sql

```
CREATE TABLE supplier_invoices (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,

    supplier_id BIGINT NOT NULL,
    invoice_number VARCHAR(255) NOT NULL,
    invoice_date DATE NOT NULL,

    invoice_amount DECIMAL(12,2) NOT NULL,
    invoice_status ENUM(
        'PENDING',
        'APPROVED',
        'REJECTED',
        'PAID'
    ) DEFAULT 'PENDING',

    notes TEXT,
    created_by BIGINT,

    FOREIGN KEY (supplier_id) REFERENCES suppliers(id),
    FOREIGN KEY (created_by) REFERENCES users(id),

    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);
```

20B.

database/migrations/034_receiving_notes.sql

```
CREATE TABLE receiving_notes (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,

    supplier_id BIGINT NOT NULL,
    supplier_invoice_id BIGINT,

    received_by BIGINT NOT NULL,
    received_date DATE NOT NULL,

    receiving_status ENUM(
        'DRAFT',
        'RECEIVED',
        'DISPUTED',
        'CANCELLED'
    ) DEFAULT 'DRAFT',

    notes TEXT,

    FOREIGN KEY (supplier_id) REFERENCES suppliers(id),
    FOREIGN KEY (supplier_invoice_id) REFERENCES supplier_invoices(id),
    FOREIGN KEY (received_by) REFERENCES users(id),

    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);
```

20C.

database/migrations/035_receiving_note_items.sql

```
CREATE TABLE receiving_note_items (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,

    receiving_note_id BIGINT NOT NULL,
    product_id BIGINT NOT NULL,

    quantity_received DECIMAL(12,2) NOT NULL,
    unit_cost DECIMAL(12,2) NOT NULL,
    total_cost DECIMAL(12,2) NOT NULL,

    FOREIGN KEY (receiving_note_id) REFERENCES receiving_notes(id),
    FOREIGN KEY (product_id) REFERENCES products(id),

    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);
```

20D.

database/migrations/036_stock_transfers.sql

```
CREATE TABLE stock_transfers (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,

    product_id BIGINT NOT NULL,

    transfer_from ENUM(
        'STORE',
        'KITCHEN',
        'BAR'
    ) NOT NULL,

    transfer_to ENUM(
        'STORE',
        'KITCHEN',
        'BAR'
    ) NOT NULL,

    quantity DECIMAL(12,2) NOT NULL,

    transfer_status ENUM(
        'PENDING',
        'APPROVED',
        'REJECTED',
        'COMPLETED'
    ) DEFAULT 'PENDING',

    requested_by BIGINT NOT NULL,
    approved_by BIGINT,
    approved_at TIMESTAMP NULL,

    notes TEXT,

    FOREIGN KEY (product_id) REFERENCES products(id),
    FOREIGN KEY (requested_by) REFERENCES users(id),
    FOREIGN KEY (approved_by) REFERENCES users(id),

    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);
```

20E. Update

backend/routes/api.php

Add routes:

```
$router->get('/api/storekeeping/supplier-invoices', [InventoryController::class, 'supplierInvoices']);
$router->post('/api/storekeeping/supplier-invoices', [InventoryController::class, 'createSupplierInvoice']);
$router->post('/api/storekeeping/supplier-invoices/{id}/approve', [InventoryController::class,
'approveSupplierInvoice']);
$router->post('/api/storekeeping/supplier-invoices/{id}/reject', [InventoryController::class,
'rejectSupplierInvoice']);

$router->get('/api/storekeeping/receiving-notes', [InventoryController::class, 'receivingNotes']);
$router->post('/api/storekeeping/receiving-notes', [InventoryController::class, 'createReceivingNote']);
$router->post('/api/storekeeping/receiving-notes/{id}/receive', [InventoryController::class, 'receiveStock']);

$router->get('/api/storekeeping/stock-transfers', [InventoryController::class, 'stockTransfers']);
$router->post('/api/storekeeping/stock-transfers', [InventoryController::class, 'createStockTransfer']);
```

```
$router->post('/api/storekeeping/stock-transfers/{id}/approve', [InventoryController::class,
'approveStockTransfer']);

$router->get('/api/storekeeping/stock-movement-report', [InventoryController::class, 'stockMovementReport']);
$router->get('/api/storekeeping/supplier-performance-report', [InventoryController::class,
'supplierPerformanceReport']);
```

20F. Update

backend/app/Controllers/InventoryController.php

Add methods inside the class:

```
public function supplierInvoices(): void
{
    Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'STOREKEEPER', 'ACCOUNTANT']);

    $stmt = Database::connection()->query(
        "SELECT
            si.*,
            s.supplier_name,
            u.full_name AS created_by_name
         FROM supplier_invoices si
         JOIN suppliers s ON s.id = si.supplier_id
         LEFT JOIN users u ON u.id = si.created_by
         ORDER BY si.created_at DESC"
    );

    Response::success(['supplier_invoices' => $stmt->fetchAll()]);
}

public function createSupplierInvoice(): void
{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'STOREKEEPER', 'ACCOUNTANT']);
    $data = Request::body();

    $stmt = Database::connection()->prepare(
        'INSERT INTO supplier_invoices
        (supplier_id, invoice_number, invoice_date, invoice_amount, notes, created_by)
        VALUES (:supplier_id, :invoice_number, :invoice_date, :invoice_amount, :notes, :created_by)'
    );

    $stmt->execute([
        'supplier_id' => $data['supplier_id'],
        'invoice_number' => $data['invoice_number'],
        'invoice_date' => $data['invoice_date'],
        'invoice_amount' => $data['invoice_amount'],
        'notes' => $data['notes'] ?? null,
        'created_by' => $user['id'],
    ]);

    AuditLogger::log(
        (int) $user['id'],
        'STOREKEEPING',
        'CREATE_SUPPLIER_INVOICE',
        null,
        null,
        $data
    );

    Response::success([], 'Supplier invoice created');
}

public function approveSupplierInvoice(int $id): void
{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'ACCOUNTANT']);

    $stmt = Database::connection()->prepare(
        "UPDATE supplier_invoices SET invoice_status = 'APPROVED' WHERE id = :id"
    );

    $stmt->execute(['id' => $id]);

    AuditLogger::log(
        (int) $user['id'],
        'STOREKEEPING',
        'APPROVE_SUPPLIER_INVOICE',
        $id
    );

    Response::success([], 'Supplier invoice approved');
}

public function rejectSupplierInvoice(int $id): void
```

```
{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'ACCOUNTANT']);

    $stmt = Database::connection()->prepare(
        "UPDATE supplier_invoices SET invoice_status = 'REJECTED' WHERE id = :id"
    );

    $stmt->execute(['id' => $id]);

    AuditLogger::log(
        (int) $user['id'],
        'STOREKEEPING',
        'REJECT_SUPPLIER_INVOICE',
        $id
    );

    Response::success([], 'Supplier invoice rejected');
}

public function receivingNotes(): void
{
    Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'STOREKEEPER', 'ACCOUNTANT']);

    $stmt = Database::connection()->query(
        "SELECT
            rn.*,
            s.supplier_name,
            u.full_name AS received_by_name
         FROM receiving_notes rn
         JOIN suppliers s ON s.id = rn.supplier_id
         JOIN users u ON u.id = rn.received_by
         ORDER BY rn.created_at DESC"
    );

    Response::success(['receiving_notes' => $stmt->fetchAll()]);
}

public function createReceivingNote(): void
{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'STOREKEEPER']);
    $data = Request::body();

    $db = Database::connection();
    $db->beginTransaction();

    $stmt = $db->prepare(
        'INSERT INTO receiving_notes
        (supplier_id, supplier_invoice_id, received_by, received_date, receiving_status, notes)
        VALUES (:supplier_id, :supplier_invoice_id, :received_by, :received_date, "DRAFT", :notes)'
    );

    $stmt->execute([
        'supplier_id' => $data['supplier_id'],
        'supplier_invoice_id' => $data['supplier_invoice_id'] ?? null,
        'received_by' => $user['id'],
        'received_date' => $data['received_date'],
        'notes' => $data['notes'] ?? null,
    ]);

    $receivingNoteId = (int) $db->lastInsertId();

    foreach ($data['items'] as $item) {
        $itemStmt = $db->prepare(
            'INSERT INTO receiving_note_items
            (receiving_note_id, product_id, quantity_received, unit_cost, total_cost)
            VALUES (:receiving_note_id, :product_id, :quantity, :unit_cost, :total_cost)'
        );

        $itemStmt->execute([
            'receiving_note_id' => $receivingNoteId,
            'product_id' => $item['product_id'],
            'quantity' => $item['quantity_received'],
            'unit_cost' => $item['unit_cost'],
            'total_cost' => $item['quantity_received'] * $item['unit_cost'],
        ]);
    }

    AuditLogger::log(
        (int) $user['id'],
        'STOREKEEPING',
        'CREATE_RECEIVING_NOTE',
        $receivingNoteId,
        null,
        $data
    );

    $db->commit();
```

```
    Response::success(['receiving_note_id' => $receivingNoteId], 'Receiving note created');
}

public function receiveStock(int $id): void
{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'STOREKEEPER']);
    $db = Database::connection();

    $db->beginTransaction();

    $itemsStmt = $db->prepare(
        'SELECT * FROM receiving_note_items WHERE receiving_note_id = :id'
    );

    $itemsStmt->execute(['id' => $id]);
    $items = $itemsStmt->fetchAll();

    foreach ($items as $item) {
        $movement = $db->prepare(
            "INSERT INTO stock_movements
            (product_id, movement_type, quantity, reference_number, notes, user_id)
            VALUES (:product_id, 'PURCHASE', :quantity, :reference, 'Stock received from receiving note',
:user_id)"
        );

        $movement->execute([
            'product_id' => $item['product_id'],
            'quantity' => $item['quantity_received'],
            'reference' => 'RN-' . $id,
            'user_id' => $user['id'],
        ]);

        $stock = $db->prepare(
            'UPDATE stock_items
             SET current_quantity = current_quantity + :quantity
             WHERE product_id = :product_id'
        );

        $stock->execute([
            'quantity' => $item['quantity_received'],
            'product_id' => $item['product_id'],
        ]);
    }

    $update = $db->prepare(
        "UPDATE receiving_notes
         SET receiving_status = 'RECEIVED'
         WHERE id = :id"
    );

    $update->execute(['id' => $id]);

    AuditLogger::log(
        (int) $user['id'],
        'STOREKEEPING',
        'RECEIVE_STOCK_FROM_NOTE',
        $id
    );

    $db->commit();

    Response::success([], 'Stock received into store');
}

public function stockTransfers(): void
{
    Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'STOREKEEPER', 'BARMAN', 'CHEF']);

    $stmt = Database::connection()->query(
        "SELECT
            st.*,
            p.product_name,
            requester.full_name AS requested_by_name,
            approver.full_name AS approved_by_name
         FROM stock_transfers st
         JOIN products p ON p.id = st.product_id
         JOIN users requester ON requester.id = st.requested_by
         LEFT JOIN users approver ON approver.id = st.approved_by
         ORDER BY st.created_at DESC"
    );

    Response::success(['stock_transfers' => $stmt->fetchAll()]);
}

public function createStockTransfer(): void
{
```

```
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'STOREKEEPER']);
    $data = Request::body();

    $stmt = Database::connection()->prepare(
        'INSERT INTO stock_transfers
        (product_id, transfer_from, transfer_to, quantity, requested_by, notes)
        VALUES (:product_id, :transfer_from, :transfer_to, :quantity, :requested_by, :notes)'
    );

    $stmt->execute([
        'product_id' => $data['product_id'],
        'transfer_from' => $data['transfer_from'],
        'transfer_to' => $data['transfer_to'],
        'quantity' => $data['quantity'],
        'requested_by' => $user['id'],
        'notes' => $data['notes'] ?? null,
    ]);

    AuditLogger::log(
        (int) $user['id'],
        'STOREKEEPING',
        'CREATE_STOCK_TRANSFER',
        (int) $data['product_id'],
        null,
        $data
    );

    Response::success([], 'Stock transfer requested');
}

public function approveStockTransfer(int $id): void
{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER']);
    $db = Database::connection();

    $db->beginTransaction();

    $stmt = $db->prepare('SELECT * FROM stock_transfers WHERE id = :id LIMIT 1');
    $stmt->execute(['id' => $id]);
    $transfer = $stmt->fetch();

    if (!$transfer) {
        Response::error('Stock transfer not found', 404);
    }

    $movementType = $transfer['transfer_to'] === 'BAR'
        ? 'STORE_TO_BAR'
        : 'STORE_TO_KITCHEN';

    $movement = $db->prepare(
        'INSERT INTO stock_movements
        (product_id, movement_type, quantity, reference_number, notes, user_id)
        VALUES (:product_id, :movement_type, :quantity, :reference, :notes, :user_id)'
    );

    $movement->execute([
        'product_id' => $transfer['product_id'],
        'movement_type' => $movementType,
        'quantity' => $transfer['quantity'],
        'reference' => 'ST-' . $id,
        'notes' => 'Approved stock transfer from ' . $transfer['transfer_from'] . ' to ' .
$transfer['transfer_to'],
        'user_id' => $user['id'],
    ]);

    $update = $db->prepare(
        "UPDATE stock_transfers
         SET transfer_status = 'COMPLETED',
             approved_by = :approved_by,
             approved_at = NOW()
         WHERE id = :id"
    );

    $update->execute([
        'approved_by' => $user['id'],
        'id' => $id,
    ]);

    AuditLogger::log(
        (int) $user['id'],
        'STOREKEEPING',
        'APPROVE_STOCK_TRANSFER',
        $id,
        null,
        $transfer
    );
```

```
    $db->commit();

    Response::success([], 'Stock transfer approved and completed');
}

public function stockMovementReport(): void
{
    Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'STOREKEEPER', 'ACCOUNTANT']);

    $stmt = Database::connection()->query(
        "SELECT
            sm.*,
            p.product_name,
            p.product_category,
            u.full_name AS user_name
         FROM stock_movements sm
         JOIN products p ON p.id = sm.product_id
         LEFT JOIN users u ON u.id = sm.user_id
         ORDER BY sm.created_at DESC"
    );

    Response::success(['stock_movement_report' => $stmt->fetchAll()]);
}

public function supplierPerformanceReport(): void
{
    Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'STOREKEEPER', 'ACCOUNTANT']);

    $stmt = Database::connection()->query(
        "SELECT
            s.id AS supplier_id,
            s.supplier_name,
            s.supplier_type,
            COUNT(DISTINCT si.id) AS invoice_count,
            COALESCE(SUM(si.invoice_amount), 0) AS total_invoice_amount,
            COUNT(DISTINCT rn.id) AS receiving_note_count
         FROM suppliers s
         LEFT JOIN supplier_invoices si ON si.supplier_id = s.id
         LEFT JOIN receiving_notes rn ON rn.supplier_id = s.id
         GROUP BY s.id, s.supplier_name, s.supplier_type
         ORDER BY total_invoice_amount DESC"
    );

    Response::success(['supplier_performance_report' => $stmt->fetchAll()]);
}
```

20G. Update

apps/admin-web/src/features/operations/api/operationsApi.js

Add methods:

```
supplierInvoices: async () => {
  const { data } = await apiClient.get('/storekeeping/supplier-invoices')
  return data.data.supplier_invoices
},

createSupplierInvoice: async (payload) => {
  const { data } = await apiClient.post('/storekeeping/supplier-invoices', payload)
  return data
},

approveSupplierInvoice: async (id) => {
  const { data } = await apiClient.post(`/storekeeping/supplier-invoices/${id}/approve`, {})
  return data
},

rejectSupplierInvoice: async (id) => {
  const { data } = await apiClient.post(`/storekeeping/supplier-invoices/${id}/reject`, {})
  return data
},

receivingNotes: async () => {
  const { data } = await apiClient.get('/storekeeping/receiving-notes')
  return data.data.receiving_notes
},

createReceivingNote: async (payload) => {
  const { data } = await apiClient.post('/storekeeping/receiving-notes', payload)
  return data
},

receiveStockFromNote: async (id) => {
  const { data } = await apiClient.post(`/storekeeping/receiving-notes/${id}/receive`, {})
```

```
  return data
},

stockTransfers: async () => {
  const { data } = await apiClient.get('/storekeeping/stock-transfers')
  return data.data.stock_transfers
},

createStockTransfer: async (payload) => {
  const { data } = await apiClient.post('/storekeeping/stock-transfers', payload)
  return data
},

approveStockTransfer: async (id) => {
  const { data } = await apiClient.post(`/storekeeping/stock-transfers/${id}/approve`, {})
  return data
},

stockMovementReport: async () => {
  const { data } = await apiClient.get('/storekeeping/stock-movement-report')
  return data.data.stock_movement_report
},

supplierPerformanceReport: async () => {
  const { data } = await apiClient.get('/storekeeping/supplier-performance-report')
  return data.data.supplier_performance_report
},
```

20H.

apps/admin-web/src/features/storekeeping/pages/SupplierInvoicesPage.jsx

```
import React from 'react'
import OperationalFormPage from '../../../components/common/OperationalFormPage'
import { operationsApi } from '../../operations/api/operationsApi'

export default function SupplierInvoicesPage() {
  return (
    <OperationalFormPage
      title="Supplier Invoices"
      subtitle="Record and approve supplier invoices"
      queryKey="supplier-invoices"
      queryFn={operationsApi.supplierInvoices}
      mutationFn={operationsApi.createSupplierInvoice}
      initialForm={{
        supplier_id: '',
        invoice_number: '',
        invoice_date: '',
        invoice_amount: '',
        notes: '',
      }}
      buildPayload={(form) => ({
        ...form,
        supplier_id: Number(form.supplier_id),
        invoice_amount: Number(form.invoice_amount),
      })}
      fields={[
        { name: 'supplier_id', label: 'Supplier ID', type: 'number' },
        { name: 'invoice_number', label: 'Invoice Number' },
        { name: 'invoice_date', label: 'Invoice Date', type: 'date' },
        { name: 'invoice_amount', label: 'Invoice Amount', type: 'number' },
        { name: 'notes', label: 'Notes', type: 'textarea' },
      ]}
      columns={[
        { key: 'supplier_name', label: 'Supplier' },
        { key: 'invoice_number', label: 'Invoice No.' },
        { key: 'invoice_date', label: 'Date' },
        { key: 'invoice_amount', label: 'Amount' },
        { key: 'invoice_status', label: 'Status' },
        { key: 'created_by_name', label: 'Created By' },
      ]}
      submitLabel="Create Invoice"
    />
  )
}
```

20I.

apps/admin-web/src/features/storekeeping/pages/ReceivingNotesPage.jsx

```
import React, { useState } from 'react'
import { useMutation, useQuery, useQueryClient } from '@tanstack/react-query'
```

```
import Page from '../../../components/ui/Page'
import Card from '../../../components/ui/Card'
import Button from '../../../components/ui/Button'
import Input from '../../../components/ui/Input'
import DataTable from '../../../components/ui/DataTable'
import LoadingBlock from '../../../components/ui/LoadingBlock'
import StatusMessage from '../../../components/ui/StatusMessage'
import { operationsApi } from '../../operations/api/operationsApi'

export default function ReceivingNotesPage() {
  const qc = useQueryClient()

  const [form, setForm] = useState({
    supplier_id: '',
    supplier_invoice_id: '',
    received_date: '',
    product_id: '',
    quantity_received: '',
    unit_cost: '',
    notes: '',
  })

  const { data, isLoading, error } = useQuery({
    queryKey: ['receiving-notes'],
    queryFn: operationsApi.receivingNotes,
  })

  const createMutation = useMutation({
    mutationFn: operationsApi.createReceivingNote,
    onSuccess: () => qc.invalidateQueries({ queryKey: ['receiving-notes'] }),
  })

  const receiveMutation = useMutation({
    mutationFn: operationsApi.receiveStockFromNote,
    onSuccess: () => qc.invalidateQueries({ queryKey: ['receiving-notes'] }),
  })

  function submit(e) {
    e.preventDefault()

    createMutation.mutate({
      supplier_id: Number(form.supplier_id),
      supplier_invoice_id: form.supplier_invoice_id ? Number(form.supplier_invoice_id) : null,
      received_date: form.received_date,
      notes: form.notes,
      items: [
        {
          product_id: Number(form.product_id),
          quantity_received: Number(form.quantity_received),
          unit_cost: Number(form.unit_cost),
        },
      ],
    })
  }

  const rows = (data || []).map((item) => ({
    ...item,
    action: <Button onClick={() => receiveMutation.mutate(item.id)}>Receive Stock</Button>,
  }))

  return (
    <Page title="Receiving Notes" subtitle="Create receiving notes and receive stock into store">
      <Card title="Create Receiving Note">
        {createMutation.error ? <StatusMessage error={createMutation.error.message} /> : null}
        <form onSubmit={submit} style={{ display: 'grid', gap: 10, maxWidth: 560 }}>
          <Input placeholder="Supplier ID" value={form.supplier_id} onChange={(e) => setForm({ ...form,
supplier_id: e.target.value })} />
          <Input placeholder="Supplier Invoice ID optional" value={form.supplier_invoice_id} onChange={(e) =>
setForm({ ...form, supplier_invoice_id: e.target.value })} />
          <Input type="date" value={form.received_date} onChange={(e) => setForm({ ...form, received_date:
e.target.value })} />
          <Input placeholder="Product ID" value={form.product_id} onChange={(e) => setForm({ ...form,
product_id: e.target.value })} />
          <Input placeholder="Quantity Received" value={form.quantity_received} onChange={(e) => setForm({
...form, quantity_received: e.target.value })} />
          <Input placeholder="Unit Cost" value={form.unit_cost} onChange={(e) => setForm({ ...form, unit_cost:
e.target.value })} />
          <Input placeholder="Notes" value={form.notes} onChange={(e) => setForm({ ...form, notes:
e.target.value })} />
          <Button type="submit">Create Receiving Note</Button>
        </form>
      </Card>

      <Card title="Receiving Notes">
        {isLoading ? <LoadingBlock label="Loading receiving notes..." /> : null}
        {error ? <StatusMessage error={error.message} /> : null}
        {!isLoading && !error ? (
```

```
          <DataTable
            columns={[
              { key: 'supplier_name', label: 'Supplier' },
              { key: 'received_by_name', label: 'Received By' },
              { key: 'received_date', label: 'Date' },
              { key: 'receiving_status', label: 'Status' },
              { key: 'action', label: 'Action' },
            ]}
            rows={rows}
          />
        ) : null}
      </Card>
    </Page>
  )
}
```

20J.

apps/admin-web/src/features/storekeeping/pages/StockTransfersPage.jsx

```
import React from 'react'
import OperationalFormPage from '../../../components/common/OperationalFormPage'
import { operationsApi } from '../../operations/api/operationsApi'

export default function StockTransfersPage() {
  return (
    <OperationalFormPage
      title="Stock Transfers"
      subtitle="Transfer stock from store to kitchen or bar"
      queryKey="stock-transfers"
      queryFn={operationsApi.stockTransfers}
      mutationFn={operationsApi.createStockTransfer}
      initialForm={{
        product_id: '',
        transfer_from: 'STORE',
        transfer_to: 'KITCHEN',
        quantity: '',
        notes: '',
      }}
      buildPayload={(form) => ({
        ...form,
        product_id: Number(form.product_id),
        quantity: Number(form.quantity),
      })}
      fields={[
        { name: 'product_id', label: 'Product ID', type: 'number' },
        {
          name: 'transfer_from',
          label: 'Transfer From',
          type: 'select',
          options: [
            { value: 'STORE', label: 'Store' },
            { value: 'KITCHEN', label: 'Kitchen' },
            { value: 'BAR', label: 'Bar' },
          ],
        },
        {
          name: 'transfer_to',
          label: 'Transfer To',
          type: 'select',
          options: [
            { value: 'STORE', label: 'Store' },
            { value: 'KITCHEN', label: 'Kitchen' },
            { value: 'BAR', label: 'Bar' },
          ],
        },
        { name: 'quantity', label: 'Quantity', type: 'number' },
        { name: 'notes', label: 'Notes', type: 'textarea' },
      ]}
      columns={[
        { key: 'product_name', label: 'Product' },
        { key: 'transfer_from', label: 'From' },
        { key: 'transfer_to', label: 'To' },
        { key: 'quantity', label: 'Quantity' },
        { key: 'transfer_status', label: 'Status' },
        { key: 'requested_by_name', label: 'Requested By' },
        { key: 'approved_by_name', label: 'Approved By' },
      ]}
      submitLabel="Request Transfer"
    />
  )
}
```

20K.

apps/admin-web/src/features/storekeeping/pages/StockMovementReportPage.jsx

```
import React from 'react'
import { useQuery } from '@tanstack/react-query'
import Page from '../../../components/ui/Page'
import Card from '../../../components/ui/Card'
import DataTable from '../../../components/ui/DataTable'
import LoadingBlock from '../../../components/ui/LoadingBlock'
import StatusMessage from '../../../components/ui/StatusMessage'
import { operationsApi } from '../../operations/api/operationsApi'

export default function StockMovementReportPage() {
  const { data, isLoading, error } = useQuery({
    queryKey: ['stock-movement-report'],
    queryFn: operationsApi.stockMovementReport,
  })

  return (
    <Page title="Stock Movement Report" subtitle="Full storekeeping stock movement visibility">
      <Card>
        {isLoading ? <LoadingBlock label="Loading stock movement report..." /> : null}
        {error ? <StatusMessage error={error.message} /> : null}

        {!isLoading && !error ? (
          <DataTable
            columns={[
              { key: 'product_name', label: 'Product' },
              { key: 'product_category', label: 'Category' },
              { key: 'movement_type', label: 'Movement' },
              { key: 'quantity', label: 'Quantity' },
              { key: 'reference_number', label: 'Reference' },
              { key: 'user_name', label: 'User' },
              { key: 'created_at', label: 'Date' },
            ]}
            rows={data || []}
          />
        ) : null}
      </Card>
    </Page>
  )
}
```

20L.

apps/admin-web/src/features/storekeeping/pages/SupplierPerformanceReportPage.jsx

```
import React from 'react'
import { useQuery } from '@tanstack/react-query'
import Page from '../../../components/ui/Page'
import Card from '../../../components/ui/Card'
import DataTable from '../../../components/ui/DataTable'
import LoadingBlock from '../../../components/ui/LoadingBlock'
import StatusMessage from '../../../components/ui/StatusMessage'
import { operationsApi } from '../../operations/api/operationsApi'

export default function SupplierPerformanceReportPage() {
  const { data, isLoading, error } = useQuery({
    queryKey: ['supplier-performance-report'],
    queryFn: operationsApi.supplierPerformanceReport,
  })

  return (
    <Page title="Supplier Performance" subtitle="Supplier invoice and receiving note performance">
      <Card>
        {isLoading ? <LoadingBlock label="Loading supplier performance..." /> : null}
        {error ? <StatusMessage error={error.message} /> : null}

        {!isLoading && !error ? (
          <DataTable
            columns={[
              { key: 'supplier_name', label: 'Supplier' },
              { key: 'supplier_type', label: 'Type' },
              { key: 'invoice_count', label: 'Invoices' },
              { key: 'total_invoice_amount', label: 'Invoice Amount' },
              { key: 'receiving_note_count', label: 'Receiving Notes' },
            ]}
            rows={data || []}
          />
        ) : null}
      </Card>
    </Page>
```

```
  )
}
```

20M. Update

apps/admin-web/src/app/routes.jsx

Add imports:

```
import SupplierInvoicesPage from '../features/storekeeping/pages/SupplierInvoicesPage'
import ReceivingNotesPage from '../features/storekeeping/pages/ReceivingNotesPage'
import StockTransfersPage from '../features/storekeeping/pages/StockTransfersPage'
import StockMovementReportPage from '../features/storekeeping/pages/StockMovementReportPage'
import SupplierPerformanceReportPage from '../features/storekeeping/pages/SupplierPerformanceReportPage'
```

Add routes:

```
<Route path="/storekeeping/supplier-invoices" element={<SupplierInvoicesPage />} />
<Route path="/storekeeping/receiving-notes" element={<ReceivingNotesPage />} />
<Route path="/storekeeping/stock-transfers" element={<StockTransfersPage />} />
<Route path="/storekeeping/stock-movement-report" element={<StockMovementReportPage />} />
<Route path="/storekeeping/supplier-performance" element={<SupplierPerformanceReportPage />} />
```

20N. Update

apps/admin-web/src/components/common/SidebarNav.jsx

Add links:

```
['Supplier Invoices', '/storekeeping/supplier-invoices'],
['Receiving Notes', '/storekeeping/receiving-notes'],
['Stock Transfers', '/storekeeping/stock-transfers'],
['Stock Movement Report', '/storekeeping/stock-movement-report'],
['Supplier Performance', '/storekeeping/supplier-performance'],
```

20O.

docs/SUPPLIER_STOREKEEPING_DEEPENING_PACK.md

```
# Batch 20 — Supplier + Storekeeping Deepening Pack

## System

Meat Lovers CIMS powered by YohPal

## Purpose

This batch deepens supplier and storekeeping control.

## Features Added

1. Supplier invoices
2. Receiving notes
3. Receiving note items
4. Stock receiving
5. Stock transfers
6. Store-to-kitchen transfers
7. Store-to-bar transfers
8. Stock movement report
9. Supplier performance report

## Supplier Invoice Control

Supplier invoices can be:

- pending
- approved
- rejected
- paid

## Receiving Notes

Receiving notes confirm what was actually received.

This helps prevent:

- fake invoices
- invoices without delivery
- stock received without supplier trace
- supplier disputes
```

```
## Stock Receiving

When receiving note is marked received:

- stock movement is created
- stock balance increases

## Stock Transfers

Stock can be transferred from:

- store to kitchen
- store to bar

Management approval completes the transfer.

## Business Control Value

This batch helps Meat Lovers stop theft by ensuring:

- supplier invoices are recorded
- receiving notes prove delivery
- stock movement is traceable
- store-to-kitchen/bar movement is controlled
- supplier performance is visible
```

Batch 20 Outcome

Supplier + storekeeping now includes:

✅ supplier invoices
✅ receiving notes
✅ purchase approval
✅ stock receiving
✅ stock transfer from store to kitchen/bar
✅ stock movement report
✅ supplier performance report

Next Smart Move

Build **Batch 21 — Production + Menu Engineering Pack**, including recipe/BOM setup, kitchen production plans, ingredient consumption, food cost per plate, gross margin per menu item, wastage control, and menu engineering report.